Lecture 02: Operators, Expressions & Statements

Objectives:

- Learn about the C# operators and how they are evaluated in expressions
- Learn the Loop Constructs
- Learn the Jump and Selection Constructs

1. C# Operators & Expressions:

C# has almost identical set of operators as Java as the table below shows.

Complex expressions can be built by combining these operators with constants, variables and method calls.

When an expression contains multiple operators, the precedence of the operators control how the operators are evaluated.

When the operators have the same precedence, their associativity determines the order of evaluation.

Except for assignment operator, all other binary operators are left-associative and are evaluated from left to right.

The assignment operator, the unary operator and the conditional operator are evaluated from right to left.

For example, the following expressions are equivalent:

1 + 2 + 3 * 4	((1 + 2) + (3 * 4))

The following table list the operators according to their precedence:

Category	Operators
Primary	`x.y f(x) a[x] x++ x-- new typeof checked unchecked`
Unary	`+ - ! ~ ++x --x (T)x`
Multiplicative	`* / %`
Additive	`+ -`
Shift	`<< >>`
Relational and type testing	`< > <= >= is as`
Equality	`== !=`

Logical AND	`&`
Logical XOR	`^`
Logical OR	`|`
Conditional AND	`&&`
Conditional OR	`||`
Conditional	`?:`
Assignment	`= *= /= %= += -= <<= >>= &= ^= |=`

The `typeof` operator is used to obtain the System.Type object for a type.

Example:

```
using System;
class Test
{
  static void Main() {
    Type[] t = {
      typeof(int),
      typeof(System.Int32),
      typeof(string),
      typeof(double[]),
    };
    for (int i = 0; i < t.Length; i++) {
      Console.WriteLine(t[i].FullName);
    }
  }
}
```

output:

```
System.Int32
System.Int32
System.String
System.Double[]
```

The `as` operator is used for type-conversion (down-casting). For example, consider the following statements:

```
Student s = new GraduateStudent(…);
GraduateStudent gs;
```

Then the following statements are equivalent – well almost.

```
gs = (GraduateStudent) s;
```

```
gs = s as GraduateStudent;
```

The only difference is when the object in `s` is not compatible with `GraduateStudent`. In that case, the first statement throws `InvalidCastException`, while the second assign null to `gs`.

The `is` operator is like the `instanceof` operator in Java. It checks if an object is compatible with a type. Example:

```
if (s is GraduateStudent)
```

```
gs = s as GraduateStudent;
```

2. Loop Constructs:

The usual **for**, **while**, and **do-while** loops exists in exactly the same syntax as in Java. Example:

while loops

```
syntax: while (expression) statement[s]
```

A 'while' loop executes a statement, or a block of statements, repeatedly until the condition specified by the boolean expression returns false.

`int a = 0;`
`while (a < 3){`
`  System.Console.WriteLine(a);`
`  a++;`
`}`
	Output:
	
	0
	1
	2

do-while loops

```
syntax: do statement[s] while (expression)
```

In 'do-while' the condition is evaluated after the block of code specified in the 'do' clause has been run. So even where the condition is initially false, the block runs once. For instance, the following code outputs '4':

`int a = 4;`
`do{`
`  System.Console.WriteLine(a);`
`  a++;`
`} while (a < 3);`
	Output:
	
	4

for loops

```
syntax: for (statement1; expression; statement2)
statement[s]3
```

Statement1 is executed before the loop is entered. The loop which

is then executed corresponds to the following **'while' loop**:

```
while (expression) {statement[s]3; statement2}
```

`for (int a = 0; a<3; a++) {`
`  System.Console.WriteLine(a);`
`}`
	Output:
	0
	1
	2

In addition, C# has the **foreach** loop which has the following syntax:

```
foreach (itemType variable1 in variable2)
  Statement[s];
```

Where, *variable2* is a collection whose items are of type `itemType` and which implements the **IEnumerable** interface.

All C# arrays implement the IEnumerable interface, so we can have a statement such as the following:

```
int[] a = {1, 3, 5, 7, 9};
foreach (int i in a)
    Console.WriteLine(i);
```

3. Jump and Selection Constructs:

C# provide the following Jump constructs for jumping out of loops, switch statements or methods.

break

The 'break' statement breaks out of the 'while' and 'for' loops and out of a case in a 'switch' statement – discussed below.

continue

The 'continue' statement can be placed in any loop structure. When it executes, it moves the program counter immediately to the next iteration of the loop. The following code example uses the 'continue'

statement to count the number of values between 1 and 100 inclusive that are not multiples of seven. At the end of the loop the variable y holds the required value.

```
int y = 0;
for (int x=1; x<=100; x++){
   if ((x % 7) == 0)
     continue;
   y++;
}
```

goto

The 'goto' statement is used to make a jump to a particular labelled part of the program code. It is also used in the 'switch' statement described below. We can use a 'goto' statement to construct a loop, as in the following example (but again, this usage is not recommended):

```
int a = 0;
start:
System.Console.WriteLine(a);
a++;
if (a < 5)
   goto start;
```

Other jump statements are:

return -- exit out of a method
throw -- throws an exception and exit out of a block

Selection Statements

C# offers the same basic types of selection statements as Java, as described below:

if - else

'If-else' statement is exactly as in java as shown by the following example:

```
if (a == 5)
    System.Console.WriteLine("A is 5");
else
    System.Console.WriteLine("A is not 5");
```

If statements can also be emulated by using the conditional operator. The conditional operator returns one of two values, depending upon the value of a boolean expression.

```
int i = (myBoolean) ? 1 : 0 ;
```

sets i to 1 if myBoolean is true, and sets i to 0 if myBoolean is false.

switch

The **switch** statement also exists in the same syntax as Java.

However, C# does not allow automatic fall through between cases, which is the default in Java if a break statement is not used.

In C# you must explicitly use a break or goto statement to indicate where control should jump to. The following shows an example:

```
int  a = 2;
switch(a) {
   case 1:
        Console.WriteLine("a>0");
        goto case 2;
   case 2:
        Console.WriteLine(" and a>1");
        break;
   default:
        Console.WriteLine("a is not set");
        break;
}
```

An exception to this rule is when a case does not specify an action as in the following example:

```
switch(a) {
    case 1:
    case 2:
        Console.WriteLine(" and a>0");
        break;
    default:
        Console.WriteLine("a is not set");
        break;
}
```

Example:
The following example shows how to use some of the control structures. It also shows how to tokenize a string in C#.

```
using System;

public class ControlStructures
{
     public static void Main()
     {
          String input;
          do
          {
               Console.Write("Type int values to add or stop to
exit: ");
               input = Console.ReadLine();
               if (input.ToLower() != "stop")
               {
                    char[] delimiters = {' ', '\t', ','};
                    String[] tokens = input.Split(delimiters);
                    int sum = 0;
                    foreach (String token in tokens)
                         sum += int.Parse(token);
                    Console.WriteLine("The sum is: "+sum);
               }
          } while (input.ToLower() != "stop");
     }
}
```

There are a number of points to observe in this example:

String tokenization is achieved by using the **Split** method of the string object. This takes an array of characters as argument and returns an array of string tokens in the string, using the characters in the character array as delimiters.

Note also that we can compare string objects using the != and == operators.

There are two types of arrays in C#. One is exactly like the one in Java. It is one dimensional, but it can be used to simulate multi-dimensional arrays, as arrays of arrays. The resulting multi-dimensional can be ragged. The declaration is exactly as in Java. It also has a **Length** (with capital L) property that returns the length of the first dimension. This type of array is sufficient for us in this course.

The other type of array is an actual multi-dimensional array which is not ragged and is declared in a slightly different way. We may not need to use this type of array in this course.